Կոնստրուկտոր, դեստրուկտոր և դասի դինամիկ անդամներ

1. Դասի անդամներ և մեթոդներ

Օբյեկտային կողմնորոշված ծրագրավորման հիմքում ընկած է այն գաղափարը, որ ծրագրում տվյալներն ու դրանց հետ կատարվող գործողությունները կարելի է ներկայացնել մեկ ամբողջության՝ դասի միջոցով: Դասը նկարագրում է, թե ինչ հատկություններ կունենան ապագա օբյեկտները և ինչ գործողություններ կկարողանան կատարել: Օրինակ՝ մեքենան կարող է ունենալ մակնիշ և արագություն, իսկ արագացնելը կարող է լինել նրա գործողություններից մեկը: Դասի ներսում հայտարարված փոփոխականները կոչվում են տվյալային անդամներ կամ դաշտեր, իսկ ֆունկցիաները՝ անդամ-ֆունկցիաներ կամ մեթոդներ: Յուրաքանչյուր օբյեկտ սովորաբար ունի ոչ ստատիկ դաշտերի սեփական արժեքները:

Դասը նկարագրում է տվյալների և գործողությունների միավորումը: Յուրաքանչյուր Car օբյեկտ ունի իր speed դաշտը: getSpeed() const մեթոդը տվյալը կարդում է՝ առանց օբյեկտը փոփոխելու:

Օրինակ

```
class Car {
private:
    string brand;
    int speed = 0;
public:
    void accelerate() { speed += 10; }
    int getSpeed() const { return speed; }
};
```

Կոդի բացատրություն

Դաշտերը պահպանում են օբյեկտի վիճակը: Մեթոդները աշխատում են այդ տվյալների հետ: private անդամներին main()-ից անմիջապես դիմել չի կարելի:

2. Կոնստրուկտորի նշանակությունը

Օբյեկտ ստեղծելիս կարևոր է, որ նրա տվյալները սկզբից իսկ ունենան որոշակի, իմաստալից արժեքներ: Այդ խնդիրը լուծում է կոնստրուկտորը: Այն հատուկ անդամ-ֆունկցիա է, որն ավտոմատ կանչվում է օբյեկտի սկզբնավորման ընթացքում: Կոնստրուկտորի անունը համընկնում է դասի անվան հետ, և այն վերադարձվող տիպ չունի՝ նույնիսկ void: Եթե մեքենայի մակնիշը դեռ հայտնի չէ, կարող ենք այն սկզբնավորել Unknown արժեքով: Այսպիսով, օբյեկտը պատրաստ է օգտագործման անմիջապես ստեղծվելուց հետո:

Car c հայտարարությունը ստեղծում է օբյեկտ և կանչում Car() կոնստրուկտորը: Այն նախնական արժեք է տալիս brand դաշտին, որպեսզի օբյեկտը լինի կանխատեսելի վիճակում:

Օրինակ

```
class Car {
    string brand;
public:
    Car() { brand = "Unknown"; }
    void show() const { cout << brand; }
};
int main() { Car c; c.show(); }
```

Կոդի բացատրություն

Կոնստրուկտորը ավտոմատ կանչվում է օբյեկտի ստեղծման պահին: Անունը նույնն է, ինչ դասինը, և վերադարձվող տիպ չունի:

3. Կոնստրուկտոր՝ պարամետրերով

Երբ բոլոր օբյեկտները չպետք է ունենան նույն սկզբնական տվյալները, օգտագործվում է պարամետրերով կոնստրուկտոր: Օբյեկտ ստեղծելիս փոխանցված արժեքները հասնում են կոնստրուկտորին և օգտագործվում են նրա դաշտերի սկզբնավորման համար: Հատուկ ուշադրություն դարձնենք սկզբնավորման ցուցակին՝ երկու կետից հետո գրված արտահայտություններին: Դրանք դաշտերը սկզբնավորում են անմիջապես՝ նախքան կոնստրուկտորի մարմնի կատարումը: Սա հատկապես կարևոր է const անդամների և հղումների դեպքում:

Սկզբնավորման ցուցակը անդամներին սկզբնավորում է անմիջապես: Դաշտերն իրականում սկզբնավորվում են դասում հայտարարված հերթականությամբ, ոչ թե ցուցակում գրված հերթականությամբ:

Օրինակ

```
class Car {
    string brand;
    int year;
public:
    Car(string b, int y) : brand(b), year(y) {}
    void show() const { cout << brand << " " << year; }
};
Car c("Toyota", 2020);
```

Կոդի բացատրություն

Պարամետրերը թույլ են տալիս ստեղծել տարբեր սկզբնական վիճակներով օբյեկտներ: : brand(b), year(y) մասը սկզբնավորման ցուցակն է:

4. Default կոնստրուկտոր

Default է կոչվում այն կոնստրուկտորը, որը հնարավոր է կանչել առանց արգումենտների: Սովորաբար այն օգտագործվում է այն դեպքերում, երբ օբյեկտը ստեղծվում է առանց լրացուցիչ տվյալների: Պետք է տարբերել երկու իրավիճակ. երբ ծրագրավորողն ինքն է գրել այդ կոնստրուկտորը, և երբ այն թույլատրելի պայմաններում ինքնաբերաբար հայտարարվում է կոմպիլյատորի կողմից: Եթե դասում արդեն գրել ենք միայն պարամետր պահանջող կոնստրուկտոր, ապա առանց արգումենտների ստեղծումը սովորաբար այլևս չի աշխատի, քանի դեռ համապատասխան կոնստրուկտոր չենք տրամադրել:

Եթե դասում հայտարարված է միայն պարամետրերով կոնստրուկտոր, կոմպիլյատորը սովորաբար ինքնուրույն default կոնստրուկտոր չի տրամադրում: `Car a();` գրառումը օբյեկտ չի ստեղծում, այլ ֆունկցիայի հայտարարություն է:

Օրինակ

```
class Car {
    int speed;
public:
    Car() : speed(0) {}
};
int main() {
    Car a;       // Car() կանչ
    Car b{};     // Car() կանչ
}
```

Կոդի բացատրություն

Default կոնստրուկտորը հնարավոր է կանչել առանց արգումենտների: Այն կարող է լինել օգտատիրոջ գրած կամ կոմպիլյատորի ստեղծած:

Կարևոր տարբերություն. default կոնստրուկտորը պարտադիր չէ, որ գրված լինի առանց պարամետրերի. այն կարող է ունենալ լռելյայն արժեքներ ունեցող բոլոր պարամետրերը: Սակայն գերբեռնման դեպքում պետք է խուսափել երկիմաստ կանչերից:

5. Կոնստրուկտորների գերբեռնում

Միևնույն դասի օբյեկտները երբեմն ստեղծվում են տարբեր քանակի տեղեկություններով: Մեկ դեպքում կարող ենք իմանալ միայն մեքենայի մակնիշը, մեկ այլ դեպքում՝ նաև արագությունը: Այդ նպատակով C++-ը թույլ է տալիս նույն դասում սահմանել մի քանի կոնստրուկտոր: Այս երևույթը կոչվում է գերբեռնում: Կոմպիլյատորը ընտրում է համապատասխան կոնստրուկտորը՝ հաշվի առնելով փոխանցված արգումենտների քանակը, տիպերը և փոխակերպման կանոնները:

Գերբեռնումը compile-time մեխանիզմ է: Վերադարձվող տիպը չի կարող տարբերակել կոնստրուկտորները, քանի որ կոնստրուկտորները վերադարձվող տիպ չունեն:

Օրինակ

```
class Car {
   string brand;
   int speed;
public:
   Car() : brand("Unknown"), speed(0) {}
   Car(string b) : brand(b), speed(0) {}
   Car(string b, int s) : brand(b), speed(s) {}
};
Car a; Car b("BMW"); Car c("Kia", 50);
```

Կոդի բացատրություն

Նույն անունով մի քանի կոնստրուկտոր տարբերվում են պարամետրերի քանակով կամ տիպերով: Ընտրվում է կանչին համապատասխան տարբերակը:

6. Գերբեռնման սխալներ

Երկու Car(int) կոնստրուկտոր ունենալ հնարավոր չէ՝ անկախ պարամետրերի անուններից: Պետք է փոխել պարամետրերի տիպերը կամ քանակը:

Օրինակ

```
class Car {
public:
   Car(int year) {}
   // Car(int speed) {} // ՍԽԱԼ՝ նույն ստորագրություն
   Car(string brand) {}
};
Car a(2020);
Car b("Ford");
```

Կոդի բացատրություն

Պարամետրի անունը չի մասնակցում գերբեռնմանը: Կարևոր են պարամետրերի տիպերը և դրանց հերթականությունը:

7. Դեստրուկտորի նշանակությունը

Օբյեկտի կյանքի ընթացքը միայն ստեղծմամբ չի ավարտվում: Երբ օբյեկտի գոյության ժամանակը լրանում է, անհրաժեշտ է կատարել նաև ավարտական գործողությունները: Դրա համար նախատեսված է դեստրուկտորը: Դրա անունը կազմվում է դասի անունից՝ սկզբում ավելացնելով ~ նշանը: Դեստրուկտորը չի ընդունում պարամետրեր և չունի վերադարձվող տիպ: Հատկապես օգտակար է այն ժամանակ, երբ օբյեկտը տնօրինում է դինամիկ հիշողություն կամ այլ ռեսուրս:

Լոկալ c օբյեկտը ոչնչացվում է main()-ի բլոկի վերջում: Դեստրուկտորը ծառայում է օբյեկտին պատկանող ռեսուրսների ազատմանը:

Օրինակ

```
class Car {
public:
```

```
    Car()  { cout << "Created\n"; }
    ~Car() { cout << "Destroyed\n"; }
};
int main() {
    Car c;
    cout << "Working\n";
}
```

Կոդի բացատրություն
Դեստրուկտորը ~ClassName() տեսքն ունի, պարամետրեր և վերադարձվող տիպ չունի: Ավտոմատ աշխատում է օբյեկտի կյանքի ավարտին:

Դեստրուկտորը հնարավոր չէ գերբեռնել: Մի դասում գոյություն ունի մեկ դեստրուկտոր: Դեստրուկտորի ավտոմատ կանչի պահը կախված է օբյեկտի պահպանման տևողությունից:

8. Ստեղծման և ոչնչացման հերթականությունը

Դասարանում կարելի է դիտարկել երկու մեքենայի օբյեկտների ստեղծումը նույն բլոկում: Նախ կկատարվի առաջինի, ապա երկրորդի կոնստրուկտորը: Երբ կատարման ընթացքը դուրս գա այդ բլոկից, տեղային օբյեկտների դեստրուկտորները սովորաբար կկանչվեն հակառակ հերթականությամբ: Այս կանոնը օգնում է հասկանալ, թե ինչու է կարևոր օբյեկտների կյանքի տևողությունը և ինչու չպետք է օգտագործել արդեն ոչնչացված օբյեկտին պատկանող տվյալները:

Սա հատկապես կարևոր է, երբ օբյեկտները ռեսուրսներ են կառավարում: Հիշել՝ ստեղծում 1, 2, ոչնչացում 2, 1:

Օրինակ
```
class Car {
    int id;
public:
    Car(int n) : id(n) { cout << "+" << id << " "; }
    ~Car() { cout << "-" << id << " "; }
};
int main() { Car a(1); Car b(2); }
// +1 +2 -2 -1
```

Կոդի բացատրություն
Նույն բլոկի լոկալ օբյեկտները ոչնչացվում են ստեղծման հակառակ հերթականությամբ:

9. this ցուցիչի իմաստը

Յուրաքանչյուր ոչ ստատիկ անդամ-ֆունկցիա աշխատում է կոնկրետ օբյեկտի հետ: C++-ում այդ օբյեկտին կարելի է դիմել this հատուկ ցուցիչի միջոցով: Այն հատկապես հարմար է, երբ մեթոդի պարամետրի անունը համընկնում է դասի դաշտի անվան հետ: Օրինակ՝ this->speed գրառումը հստակ ցույց է տալիս, որ

խոսքը ընթացիկ օբյեկտի speed դաշտի մասին է, մինչդեռ պարզապես speed-ը կարող է նշանակել նույնանուն պարամետրը:

Պարամետրի speed անունը ծածկում է դաշտի նույն անունը: this->speed-ը վերաբերում է տվյալ օբյեկտի դաշտին, իսկ աջ կողմի speed-ը՝ մեթոդի պարամետրին:

Օրինակ

```
class Car {
   int speed;
public:
   void setSpeed(int speed) {
      this->speed = speed;
   }
};
```

Կոդի բացատրություն

this-ը ցույց է տալիս այն օբյեկտը, որի համար տվյալ պահին կանչվել է ոչ ստատիկ մեթոդը:

10. this և *this

this-ը ցուցիչ է ընթացիկ օբյեկտին, իսկ *this-ը այդ ցուցիչի միջոցով ստացված հենց օբյեկտն է: Եթե մեթոդը վերադարձնում է Car& տիպ, կարող ենք վերադարձնել *this և թույլ տալ մեթոդների շղթայական կանչեր: Կարևոր է հասկանալ, որ սա նոր օբյեկտ չի ստեղծում. վերադարձվում է արդեն գոյություն ունեցող օբյեկտի հղումը:

this-ի տիպը Car* է (const մեթոդում՝ const Car*): *this-ը օբյեկտի հղումն է: Չպետք է վերադարձնել հղում արդեն ոչնչացված տեղային օբյեկտին:

Օրինակ

```
class Car {
   int speed = 0;
public:
   Car& accelerate(int value) {
      this->speed += value;
      return *this;
   }
};
Car c;
c.accelerate(10).accelerate(20);
```

Կոդի բացատրություն

this-ը ցուցիչ է, իսկ *this-ը տվյալ օբյեկտն է: Car& վերադարձը թույլ է տալիս շղթայական կանչեր:

11. Դինամիկ հիշողություն՝ new

Հատկացված հիշողությունը չի ազատվում միայն ցուցիչի տեսանելիության ավարտով: Եթե delete-ը բաց թողնենք, կարող է առաջանալ հիշողության արտահոսք:

Օրինակ

```
int* speed = new int(60);
cout << *speed;   // 60
delete speed;
speed = nullptr;
```

Կոդի բացատրություն

new-ը հիշողություն է հատկացնում դինամիկ տիրույթում և վերադարձնում հասցե: delete-ը ազատում է համապատասխան հիշողությունը:

12. Դասի դինամիկ անդամ

Սա ուսուցողական օրինակ է: Իրական ծրագրերում մեկ int-ի համար նախընտրելի է սովորական int դաշտը: Սեփականող հում ցուցիչը պահանջում է պատճենման կանոնների հատուկ ուշադրություն:

Օրինակ

```
class Car {
    int* speed;
public:
    Car(int s) : speed(new int(s)) {}
    void show() const { cout << *speed; }
    ~Car() { delete speed; }
};
```

Կոդի բացատրություն

speed-ը ցուցիչ անդամ է, որը պահում է դինամիկ հիշողության հասցեն: Կոնստրուկտորը հատկացնում է, դեստրուկտորը՝ ազատում:

Կոնստրուկտորում new-ի հաջող կատարման դեպքում օբյեկտը տիրապետում է ռեսուրսին: Եթե դասը պատճենվում է, անհրաժեշտ է վերահսկել պատճենման իմաստը:

13. Դինամիկ զանգված

speeds ցուցիչը պահում է զանգվածի առաջին տարրի հասցեն: Օրինակում speeds[1]-ը 20 է: Ավելի անվտանգ իրական լուծումը std::vector<int>-ն է:

Օրինակ

```
class Car {
    int* speeds;
public:
    Car() : speeds(new int[3]{10, 20, 30}) {}
    void show() const { cout << speeds[1]; }
```

```
    ~Car() { delete[] speeds; }
};
```

Կոդի բացատրություն

new[]-ով ստեղծված զանգվածը պարտադիր ազատել delete[]-ով: new/delete և new[]/delete[] զույգերը չեն փոխարինում իրար: Հիշողության կառավարման ժամանակ պետք է տարբերել ցուցիչը և նրա մատնանշած տվյալը: Ցուցիչը պահում է հասցե, իսկ հատկացված հիշողությունը շարունակում է գոյություն ունենալ, մինչև այն ճիշտ ձևով ազատվի: Եթե նույն հասցեն ազատենք երկու անգամ կամ կորցնենք վերջին ցուցիչը՝ առանց ազատելու հիշողությունը, ծրագիրը կարող է աշխատել սխալ:

14. Սխալ՝ մակերեսային պատճենում

Սա double delete սխալի օրինակ է: Մինչև պատճենող կոնստրուկտորը և վերագրման օպերատորը ճիշտ սահմանելը նման օբյեկտը պատճենել չի կարելի: Հիշողության կառավարման ժամանակ պետք է տարբերել ցուցիչը և նրա մատնանշած տվյալը: Ցուցիչը պահում է հասցե, իսկ հատկացված հիշողությունը շարունակում է գոյություն ունենալ, մինչև այն ճիշտ ձևով ազատվի: Եթե նույն հասցեն ազատենք երկու անգամ կամ կորցնենք վերջին ցուցիչը՝ առանց ազատելու հիշողությունը, ծրագիրը կարող է աշխատել սխալ:

Օրինակ

```
class Car {
    int* speed;
public:
    Car(int s) : speed(new int(s)) {}
    ~Car() { delete speed; }
};
// Car a(30);
// Car b = a;  // ՎՏԱՆԳԱՎՈՐ
```

Կոդի բացատրություն

Լռելյայն պատճենումը պատճենում է ցուցիչի հասցեն, ոչ թե հասցեում գտնվող տվյալը: Երկու դեստրուկտոր կարող են փորձել ազատել նույն հիշողությունը:

Ուշադրություն. նշված վտանգավոր տողերը դիտմամբ մեկնաբանված են: Եթե դրանք գործարկվեն, ծրագիրը կարող է ունենալ չսահմանված վարք:

15. Խորքային պատճենում

Այս օրինակը լուծում է պատճենման կառուցման խնդիրը, բայց ամբողջական չէ. պետք է ճիշտ սահմանել նաև պատճենման վերագրման օպերատորը կամ արգելել վերագրումը:

Օրինակ

```
class Car {
    int* speed;
public:
    Car(int s) : speed(new int(s)) {}
    Car(const Car& other)
        : speed(new int(*other.speed)) {}
    ~Car() { delete speed; }
};
```

Կոդի բացատրություն

Պատճենող կոնստրուկտորը նոր հիշողություն է հատկացնում և պատճենում արժեքը: Օբյեկտներն այլևս նույն հասցեն չեն կիսում:

16. Rule of Three

Այս օրինակում բոլոր Car օբյեկտներն ունեն հատկացված մեկ int, հետևաբար վերագրման ժամանակ կարելի է փոխել *speed արժեքը: Ինքնավերագրման ստուգումը this != &x է:

Օրինակ

```
class Car {
    int* speed;
public:
    Car(int s) : speed(new int(s)) {}
    Car(const Car& x) : speed(new int(*x.speed)) {}
    Car& operator=(const Car& x) {
        if (this != &x) *speed = *x.speed;
        return *this;
    }
    ~Car() { delete speed; }
};
```

Կոդի բացատրություն

Եթե դասը կառավարում է սեփական հում ցուցիչ, հաճախ անհրաժեշտ են երեք հատուկ անդամ՝ դեստրուկտոր, պատճենող կոնստրուկտոր, պատճենման վերագրում:

Rule of Three-ի այս պարզ օրինակում *speed-ը միշտ վավեր է, քանի որ յուրաքանչյուր կոնստրուկտոր հատկացնում է հիշողություն: Ավելի բարդ դասերի դեպքում վերագրման համար հաճախ կիրառվում է copy-and-swap տեխնիկան:

17. Ավելի անվտանգ մոտեցում

Սա Rule of Zero մոտեցման օրինակ է. ռեսուրսների կառավարումը վստահում ենք ստանդարտ գրադարանի տիպերին: Անհրաժեշտ է #include <vector>:

Օրինակ

```
class Car {
    vector<int> speeds;
```

```
public:
    Car() : speeds{10, 20, 30} {}
    void add(int x) { speeds.push_back(x); }
    int count() const { return (int)speeds.size(); }
};
```

Կոդի բացատրություն

std::vector-ը ինքնուրույն կառավարում է զանգվածի հիշողությունը: Սովորաբար ձեռքով new[] և delete[] պետք չեն:

18. Ամփոփում և ստուգման հարցեր

Քննարկման հարցեր՝ ինչո՞վ է կոնստրուկտորը տարբերվում սովորական մեթոդից, ինչո՞ւ է վտանգավոր հում ցուցիչի մակերեսային պատճենումը, ո՞ր դեպքերում ընտրել std::vector:

Օրինակ

```
Car a;
Car b("BMW");
Car c("Toyota", 80);
// Ո՞ր կոնստրուկտորն է կանչվում:
// Ե՞րբ է կանչվում դեստրուկտորը:
// Ինչո՞ւ է պետք this->speed:
// new[]-ին ո՞ր delete-ն է համապատասխանում:
```

Կոդի բացատրություն

Վերհիշել օբյեկտի կյանքի փուլերը՝ ստեղծում, սկզբնավորում, մեթոդների կանչ, ոչնչացում:

Ուսանողների ինքնաստուգման առաջադրանքներ

1. Գրել Car դաս՝ երեք տարբեր կոնստրուկտորներով և show() մեթոդով:
2. Բացատրել, թե երբ է աշխատում դեստրուկտորը երկու լոկալ օբյեկտների համար:
3. Գրել setSpeed(int speed) մեթոդ՝ օգտագործելով this->speed:
4. Ստեղծել երեք տարրով դինամիկ զանգված և ճիշտ ազատել հիշողությունը:
5. Բացատրել՝ ինչու է Car b = a գրառումը վտանգավոր հում ցուցիչի դեպքում:
6. Փոխարինել դինամիկ զանգվածը std::vector-ով: